GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 24 OF 42 | MORE PRACTICE CHAPTER TOPIC

More practice - Hash maps and sets

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Hash maps and sets | 5 questions | tap a title to jump to it

M	1. Find a Horizontal Cut That Bisects Rectangle Area	PracHub	2026-06-04
M	2. Website Activity Analytics: Unique Users + Session Time	1Point3Acres	2026-06-26
M	3. Check pivot after removing one element	PracHub	2026-02-12
M	4. Reconstruct original array from doubled shuffle	PracHub	2026-01-04
M	5. Gold Chain Split into Two Equal Halves	1Point3Acres	2026-04-21

STUDY PATH: question 94 of 167 (more practice)

Priority score 2.82 | Hash maps and sets | asked 138 days ago

MEDIUM | PracHub | Technical Screen | 1 of 5

Find a Horizontal Cut That Bisects Rectangle Area

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 8.1** **30 DAYS 0** **PRIORITY 169/228**

ABOUT THIS QUESTION

Topic: More practice - Hash maps and sets

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #850 Rectangle Area II

LeetCode match: No direct match; related LC 850 Rectangle Area II (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-08-07)

Original: https://prachub.com/coding-questions/find-a-horizontal-cut-that-bisects-rectangle-area

QUESTION

Overview: Solve a computational-geometry problem that asks for the horizontal line dividing the combined area of many disjoint rectangles in half. The exercise emphasizes numerical precision, large coordinate ranges, performance at 200,000 rectangles, and careful treatment of partially intersected shapes.

APPROACH

Compute total area and the covered width per y-slab via a sweep; walk slabs accumulating area until half, then interpolate inside the slab. Use exact arithmetic for precision.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the rectangles are disjoint (the overview says so), the coordinate range, whether the answer must be exact (use fractions) and which y to return if a gap makes many cuts valid (the lowest).

BRUTE FORCE TO SAY OUT LOUD

Binary search on y, summing the rectangle area below y each time. O(n log precision), but floating point error is a risk.

OPTIMAL IDEA

Because the rectangles are disjoint, the area below y is a piecewise linear function whose slope is the sum of the widths of the rectangles spanning y. Create events at each rectangle's bottom (slope increases by its width) and top (slope decreases), sort by y, and walk the slabs accumulating area. When the next slab would pass half of the total, solve inside it: y = y0 + (half - accumulated) / slope, using integer or Fraction arithmetic. O(n log n) for 200,000 rectangles.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

One rectangle (the middle), two rectangles separated by a gap (slope zero there), the half exactly at a rectangle boundary, huge coordinates.

LIKELY FOLLOW-UPS

Overlapping rectangles (LeetCode 850, needs a segment tree over x), or a vertical cut.

PITFALLS

Floating point error when comparing areas. Not handling slabs with slope zero.

HOW TO THINK ABOUT IT

Area below a line y = c is a piecewise linear function of c, and between two consecutive rectangle edges its slope is just the total width of rectangles crossing that height (they are disjoint, so widths add). Walk the slabs between events, accumulate area, and when half the area falls inside a slab solve a linear equation. Use exact fractions so the answer is exact.

Basis: Written; a sweep over y events (compare LeetCode 850, Rectangle Area II, which is the harder overlapping version). Assumed rule: axis-aligned rectangles (x1, y1, x2, y2) that do not overlap; return the lowest y with half the total area below it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(rects):
    total = sum(area(r) for r in rects)
    lo = lowest_edge(rects)
    hi = highest_edge(rects)
    for _ in range(100):
        mid = (lo + hi) / 2
        if area_below(rects, mid) >= total / 2:
            hi = mid
        else:
            lo = mid
    return hi
```

TIME	SPACE
O(R * iterations)	O(1)

WHY IT IS SLOW

Each step sums over all rectangles and the answer is only as precise as floating point allows. A sweep over sorted edges with exact fractions finds the slab and the cut directly in O(R log R).

OPTIMAL SOLUTION · Sweep the y events and interpolate inside one slab

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

FOLLOW-UP 2 Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

FOLLOW-UP 3 A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

FOLLOW-UP 4 A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by p * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find a Horizontal Cut That Bisects
# Rectangle Area
#
# Solve a computational-geometry problem that asks for the
# horizontal line dividing the combined area of many
# disjoint rectangles in half. The exercise emphasizes
# numerical precision, large coordinate ranges,
# performance at 200,000 rectangles, and careful treatment
# of partially intersected shapes.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rectangles may overlap (LeetCode 850).
# 2. Coordinates up to 10^9 and 200,000 rectangles.
# 3. A vertical cut instead, or a cut at a given ratio.
# 4. A cut that leaves a given fraction of the area below
# it, not exactly half.
#
# INTERVIEW NOTES
# Area below a line y = c is a piecewise linear function
# of c, and between two consecutive rectangle edges its
# slope is just the total width of rectangles crossing
# that height (they are disjoint, so widths add). Walk the
# slabs between events, accumulate area, and when half the
# area falls inside a slab solve a linear equation. Use
# exact fractions so the answer is exact.
#
# BRUTE FORCE (say it first): Binary search on the cut
# height
# The area below y is non-decreasing in y, so bisect y
# between the lowest and highest edge until the area below
# is half the total. It is simple and general, but it
# works in floating point (limited precision) and costs a
# full pass over all rectangles per step. Time O(R *
# iterations). Space O(1).


from collections import defaultdict
from fractions import Fraction


# BRUTE FORCE (say it first): Binary search on the cut
# height | time O(R * iterations)
def _area_below(rects, y):
    return sum(
        (x2 - x1) * max(0, min(y, y2) - y1)
        for x1, y1, x2, y2 in rects
    )


# HELPERS for the base question
def total_area(rects):
    # rects are disjoint (x1, y1, x2, y2)
    return sum((x2 - x1) * (y2 - y1)
               for x1, y1, x2, y2 in rects)


def bisect_cut_search(rects, iterations=100):
    total = sum(
        (x2 - x1) * (y2 - y1)
        for x1, y1, x2, y2 in rects
    )
    lo = min(r[1] for r in rects)
    hi = max(r[3] for r in rects)
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if 2 * _area_below(rects, mid) >= total:
            hi = mid
        else:
            lo = mid
    return hi


# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths.
# FOLLOW-UP 4: A cut that leaves a given fraction of the
# area below it, not exactly half.
# Answer: Replace 'half of the total' by p * total in the
# slab test; the interpolation formula inside the slab is
# the same. The code returns an exact Fraction and is
# checked by computing the area below the returned cut.
def cut_height(rects, share=Fraction(1, 2)):
    # FOLLOW-UP 4: share= leaves any fraction below the
    # cut
    # the height with `share` of the total area below it
    # (1/2 = bisect). Exact Fractions, never floats.
    want = share * total_area(rects)
    # width gained when the sweep passes each height
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    ys = sorted(events)
    width, so_far = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        # inside this slab the covered width is constant
        width += events[y0]
        gain = width * (y1 - y0)
        if width and so_far + gain >= want:
```

```
            # the cut is inside this slab: interpolate
            return y0 + (want - so_far) / width
        so_far += gain
    return Fraction(ys[-1])


# HELPERS for the checks below
def area_below(rects, y):
    # exact area under the line at height y (used to
    # check)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
            total += (x2 - x1) * (top - y1)
    return total


if __name__ == "__main__":
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    assert cut_height(stacked) == 1
    # one tall rectangle: halfway up
    assert cut_height([(0, 0, 1, 4)]) == 2
    # a gap between rectangles: the cut sits at the first
    # top
    gap = [(0, 0, 1, 1), (0, 5, 1, 6)]
    assert cut_height(gap) == 1
    # the cut can fall inside a slab with a fractional
    # height
    odd = [(0, 0, 3, 1), (0, 1, 1, 2)]
    y = cut_height(odd)
    assert area_below(odd, y) * 2 == total_area(odd)
    # a different share: 1/4 of the area below
```

```
    q = cut_height(stacked, Fraction(1, 4))
    below = area_below(stacked, q)
    assert below * 4 == total_area(stacked)
    # large coordinates stay exact
    big = [(0, 0, 10 ** 9, 10 ** 9)]
    assert cut_height(big) == Fraction(10 ** 9, 2)
```

FUNCTION AND TEST CASES

Signature

```
def horizontal_bisector(rectangles: list[list[float]]) ->
float:
```

Sample call

```
horizontal_bisector([[0, 0, 4, 2]])
```

Test cases

Input	Expected
`([[0, 0, 4, 2]],)`	`1.0`
`([[0, 0, 2, 4], [10, 1, 2, 2]],)`	`2.0`

ALSO REPORTED · 1 more

Find where to cut cake into equal areas | PracHub | Technical Screen

https://prachub.com/coding-questions/find-where-to-cut-cake-into-equal-areas

STUDY PATH: question 95 of 167 (more practice)

Priority score 2.5 | Hash maps and sets | asked 116 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 2 of 5

Website Activity Analytics: Unique Users + Session Time

ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 217/228**

ABOUT THIS QUESTION

Topic: More practice - Hash maps and sets

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

Original: https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

QUESTION

Given website action records with **`user_id`**, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code.

Input is a collection of user-activity records. Each record contains:

user_id: numeric user identifier
timestamp: timestamp string such as 2024-07-26 10:00:00
activity: action name such as login, view, purchase, or logout

Implement two analytics tasks

Return a mapping from each activity type to the number of unique users who performed that activity.
Calculate the average time spent on the website per user in seconds.

Clarify session semantics before coding. The prompt uses login/logout-style events, but the exact treatment of users without an explicit logout, multiple sessions per user, and out-of-order records must be agreed with the interviewer.

EXAMPLES

```
user_activity = [
    {'user_id': 1, 'timestamp': '2024-07-26
        10:00:00', 'activity': 'login'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:05:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:10:00', 'activity': 'view'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:15:00', 'activity': 'purchase'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:20:00', 'activity': 'view'},
    {'user_id': 3, 'timestamp': '2024-07-26
        10:25:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:30:00', 'activity': 'logout'},
]
```

APPROACH

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

HOW TO THINK ABOUT IT

Unique users per activity is a map from activity to a set of user ids. Time on site needs a definition: pair each login with the next logout of the same user, and say what you do for a login without a logout (here: the session ends at that user's last event). Parse timestamps once, sort each user's events by time, and agree on out-of-order rows and repeated logins before coding.

Basis: Written for this question (a grouping and aggregation task; the SQL half of the screen is the same idea with GROUP BY and COUNT(DISTINCT)).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

TIME	SPACE
O(U * N + A * N)	O(U + A)

WHY IT IS SLOW

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

OPTIMAL SOLUTION · One pass to group, then sessions per user

TIME	SPACE
O(N log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Website Activity Analytics: Unique Users
# + Session Time
#
# Given website action records with user_id, timestamp,
# and activity, compute per-activity unique-user counts
# and average time spent on the site per user in seconds.
# The screen paired this Python task with a lighter SQL
# section and emphasized edge-case discussion over perfect
# final code. Input is a collection of user-activity
# records. Each record contains: user_id: numeric user
# identifier timestamp: timestamp string such as
# 2024-07-26 10:00:00 activity: action name such as login,
# view, purchase, or logout Implement two analytics tasks:
# Return a mapping from each activity type to the number
# of unique users who performed that activity. Calculate
# the average time spent on the website per user in
# seconds. Clarify session semantics before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Users with several sessions, missing logouts or out-
# of-order rows.
#
# INTERVIEW NOTES
# Unique users per activity is a map from activity to a
# set of user ids. Time on site needs a definition: pair
# each login with the next logout of the same user, and
# say what you do for a login without a logout (here: the
# session ends at that user's last event). Parse
# timestamps once, sort each user's events by time, and
# agree on out-of-order rows and repeated logins before
# coding.
#
# BRUTE FORCE (say it first): Rescan all records for every
# user and every activity
# For each activity loop over all records to collect
# users, and for each user loop over all records again to
# compute the session time. Correct, but it rescans the
# data many times. Time O(U * N + A * N). Space O(U + A).


from collections import defaultdict
from datetime import datetime


# BRUTE FORCE (say it first): Rescan all records for every
```

```
# user and every activity | time O(U * N + A * N)
def activity_brute(records):
    from datetime import datetime

    ts = lambda r: datetime.strptime(
        r["timestamp"], "%Y-%m-%d %H:%M:%S"
    )
    users = sorted({r["user_id"] for r in records})
    unique = {
        a: len(
            {
                r["user_id"]
                for r in records
                if r["activity"] == a
            }
        )
        for a in sorted(
            {r["activity"] for r in records}
        )
    }
    total = 0
    for u in users:
        evs = sorted(
            (ts(r), r["activity"])
            for r in records
            if r["user_id"] == u
        )
        start, secs = None, 0
        for t, act in evs:
            if act == "login" and start is None:
                start = t
            elif act == "logout" and start is not None:
                secs += (t - start).total_seconds()
                start = None
        if start is not None:
            secs += (evs[-1][0] - start).total_seconds()
        total += secs
    return unique, (
        total / len(users) if users else 0.0
    )


# HELPERS for the base question
FORMAT = "%Y-%m-%d %H:%M:%S"
```

```
def when(record):
    # timestamp text -> datetime
    return datetime.strptime(record["timestamp"], FORMAT)


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
# FOLLOW-UP 1: Users with several sessions, missing
# logouts or out-of-order rows.
# Answer: Sort each user's events by time before pairing;
# a second login before a logout is ignored (or closes the
# old session, depending on the contract you agree).
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
```

```
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total
```

```
# AVERAGE time on the site per user
def average_time_per_user(records):
    events = defaultdict(list)
    for r in records:
        item = (when(r), r["activity"])
        events[r["user_id"]].append(item)
    if not events:
        return 0.0
    # state the denominator: every user who appears
    per_user = events.values()
    total = sum(session_seconds(e) for e in per_user)
    return total / len(events)


if __name__ == "__main__":
    def rec(user, minute, activity):
        stamp = f"2024-07-26 10:{minute:02}:00"
        return {"user_id": user, "activity": activity,
                "timestamp": stamp}

    data = [rec(1, 0, "login"), rec(2, 5, "login"),
            rec(1, 10, "view"), rec(1, 15, "purchase"),
            rec(2, 20, "view"), rec(3, 25, "login"),
            rec(1, 30, "logout")]
    assert unique_users_by_activity(data) == {
        "login": 3, "logout": 1, "purchase": 1, "view": 2}
    # user 1: 1800 s, user 2: 900 s (no logout), user 3: 0
    assert average_time_per_user(data) == 900
    assert average_time_per_user([]) == 0.0
    # out-of-order rows are sorted first
    shuffled = [rec(1, 30, "logout"), rec(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800
    # two sessions add up
    two = [rec(1, 0, "login"), rec(1, 5, "logout"),
           rec(1, 10, "login"), rec(1, 20, "logout")]
    assert average_time_per_user(two) == 900
```

NOTES FROM CANDIDATES

SQL in the same screen was described as simple and covered joins / aggregation style work, but the exact SQL prompt did not leak.
The interviewer accepted a strong draft when all edge cases were discussed, even though the final code did not cover every case.
Spend time up front on timestamp parsing, ordering, repeated login/logout events, and users with missing terminal actions.

PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

STUDY PATH: question 96 of 167 (more practice)

Priority score 2.33 | Hash maps and sets | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 3 of 5

Check pivot after removing one element

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 116/228

ABOUT THIS QUESTION

Topic: More practice - Hash maps and sets

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #724 Find Pivot Index

LeetCode match: LC 724 Find Pivot Index (variant: one removal) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/check-pivot-after-removing-one-element

QUESTION

Overview: This question evaluates array-manipulation skills, cumulative-sum/prefix-sum reasoning, and robustness in handling negative values and duplicates, testing algorithmic problem-solving competency in the Coding & Algorithms domain and focusing on practical application rather than purely conceptual understanding.

APPROACH

Prefix sums plus a hash map of left-side sums; for each removal candidate, check whether some index has equal left and right sums after the removal.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is not complete in my copy, so confirm: after removing exactly one element (or at most one), does there exist an index where the sum to the left equals the sum to the right (the pivot itself excluded)? Are negatives and duplicates allowed?

BRUTE FORCE TO SAY OUT LOUD

For each element removed, rebuild the array and test every pivot with prefix sums. O(n^2).

OPTIMAL IDEA

Let P[i] be the sum before index i and S the total. If the removed element r is to the right of the pivot i, the two sides are equal exactly when a[r] = S - 2 * P[i] - a[i]. If r is to the left of i, the condition is a[r] = 2 * P[i] + a[i]

INTERVIEW-DAY PLAN

- S. So for every pivot candidate the value that must be removed is determined; look it up in a dict of value to last index (needs an index greater than i) and a dict of value to first index (needs an index less than i). One pass, O(n) time and space. Say the brute force first, and mention you checked the formulas against it on random small arrays.

DRY-RUN THESE CASES

A one-element array, two elements, negatives, duplicate values, a pivot that exists without any removal, removing the pivot itself (not allowed by most readings).

LIKELY FOLLOW-UPS

Allow at most one removal or remove a pivot candidate, or count all pairs.

PITFALLS

Letting the removed element be the pivot. Forgetting that the prefix sums shift after a removal.

HOW TO THINK ABOUT IT

Use prefix sums. For a pivot at original index i and a removed element on one side, the balance equation fixes the value that has to be removed: it is a closed form in the prefix sum, the total and a[i]. So for every candidate pivot you only have to ask 'does that value occur on the correct side?', which a dictionary of first and last positions answers in O(1).

Basis: Variant of LeetCode 724 (Find Pivot Index). Assumed rule (confirm it): remove exactly one element; is there then an index where the sum on its left equals the sum on its right (the pivot excluded)?

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    for r in range(len(a)):
        b = a[:r] + a[r + 1:]
        for i in range(len(b)):
            if sum(b[:i]) == sum(b[i + 1:]):
                return True
    return False
```

TIME	SPACE
O(n^3)	O(n)
O(n^2) with prefix sums	

WHY IT IS SLOW

n removals times n pivots times a sum: O(n^3) (O(n^2) with prefix sums). For each pivot the value to remove is determined by prefix sums, so a dictionary lookup makes it O(n).

OPTIMAL SOLUTION · Closed-form value to remove, found in a dictionary of positions

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

FOLLOW-UP 2 At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

FOLLOW-UP 3 Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check pivot after removing one element
#
# This question evaluates array-manipulation skills,
# cumulative-sum/prefix-sum reasoning, and robustness in
# handling negative values and duplicates, testing
# algorithmic problem-solving competency in the Coding &
# Algorithms domain and focusing on practical application
# rather than purely conceptual understanding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Without any removal (LeetCode 724).
# 2. At most one removal (removal optional).
# 3. Return which element to remove and the pivot.
#
# INTERVIEW NOTES
# Use prefix sums. For a pivot at original index i and a
# removed element on one side, the balance equation fixes
# the value that has to be removed: it is a closed form in
# the prefix sum, the total and a[i]. So for every
# candidate pivot you only have to ask 'does that value
# occur on the correct side?', which a dictionary of first
# and last positions answers in O(1).
#
# BRUTE FORCE (say it first): Try every removal and every
# pivot
# For each removed index build the shorter array and test
# every pivot with sums. Obviously correct and the oracle.
# Time O(n^3), O(n^2) with prefix sums. Space O(n).


# BRUTE FORCE (checker)
def brute(a):
    # try every removal and every pivot: the demo's check
    for i in range(len(a)):
        rest = a[:i] + a[i + 1:]
        for j in range(len(rest)):
            if sum(rest[:j]) == sum(rest[j + 1:]):
                return True
    return False


# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
def pivot_after_removal(a):
    # remove ONE element, then is there a pivot: an
    # element
    # whose left sum equals its right sum? Returns
    # (removed
    # index, pivot index) or None. O(n) with two scans.
    n, total = len(a), sum(a)
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    # key[j]: left + left + a[j] when nothing is removed
    key = [2 * prefix[j] + a[j] for j in range(n)]
    # pivot j AFTER the removed index i:
    #   2 * prefix[j] - a[i] = total - a[j]  -> a[i] = key
    # - total
    where, ok = {}, None
    for j in range(n):
        i = where.get(key[j] - total)
        if i is not None:
            return i, j
        where[a[j]] = j
    # pivot j BEFORE the removed index i:
    #   2 * prefix[j] + a[j] = total - a[i]  -> a[i] =
    # total - key
    where = {}
    for j in range(n - 1, -1, -1):
        i = where.get(total - key[j])
        if i is not None:
            return i, j
        where[a[j]] = j
    return ok


# FOLLOW-UP 1: Without any removal (LeetCode 724).
# Answer: One pass: the pivot is the index where left sum
# equals total minus left minus the element. O(n), O(1).
def pivot_index(a):
    # LeetCode 724: no removal, O(1) extra space
    total, left = sum(a), 0
    for i, x in enumerate(a):
        if left == total - left - x:
            return i
        left += x
    return -1


# FOLLOW-UP 2: At most one removal (removal optional).
# Answer: Answer true if pivot_index finds a pivot,
# otherwise run the removal test above.
def has_pivot(a, allow_removal=True):
    # follow-up: removal is optional
    if pivot_index(a) != -1:
        return True
    if not allow_removal:
        return False
    return pivot_after_removal(a) is not None


if __name__ == "__main__":
    # [1,7,3,6,5,6] has a pivot at index 3 without
    # removal,
    # but removing one element never gives another
```

```
    assert pivot_after_removal([1, 7, 3, 6, 5, 6]) is None
    # [1, 1, 1, 1] has no pivot, but removing the element
    # at
    # index 1 leaves [1, 1, 1] whose middle element is one
    assert pivot_index([1, 1, 1, 1]) == -1
    assert pivot_after_removal([1, 1, 1, 1]) == (1, 2)
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1
    assert not has_pivot([1, 2, 3], allow_removal=False)
    # check against the brute force on many small arrays
    import itertools
    for n in range(1, 6):
        every = itertools.product([-2, 0, 1, 3], repeat=n)
        for a in every:
            a = list(a)
            found = pivot_after_removal(a) is not None
            assert found == brute(a)
    assert has_pivot([1, 1, 1, 1])
    ones = [1, 1, 1, 1]
    assert not has_pivot(ones, allow_removal=False)
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> bool:
```

Sample call

```
solution([1, 2])
```

Test cases

Input	Expected
`([1, 2],)`	`True`
`([2, 1, 1, 2],)`	`True`

STUDY PATH: question 97 of 167 (more practice)

Priority score 2.19 | Hash maps and sets | asked 289 days ago

MEDIUM | PracHub | Technical Screen | 4 of 5

Reconstruct original array from doubled shuffle

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 127/228

ABOUT THIS QUESTION

Topic: More practice - Hash maps and sets

Optimal answer from: the interview site's own solution

LeetCode match: LC 2007 Find Original Array From Doubled Array (inferred from overview)

Posted on PracHub: 2026-01-04 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/reconstruct-original-array-from-doubled-shuffle

QUESTION

Overview: This question evaluates array manipulation, multiset/frequency reasoning, pairing logic, and careful handling of edge cases such as zeros and negative numbers.

APPROACH

Sort (by absolute value) and use a counter: for each smallest x with count > 0, pair with 2x; handle zeros (need even count) and negatives. O(n log n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the output may be in any order, whether negatives and zeros are possible, and what to return if it is impossible (empty array).

BRUTE FORCE TO SAY OUT LOUD

Try every pairing recursively. Exponential.

OPTIMAL IDEA

If the length is odd return empty. Count values. Process numbers in order of absolute value; for each value x with remaining count, match it with 2x, decrement both counts, and fail if 2x has no count left. Zeros need an even count. Sorting by abs makes the smallest value always the original, never a double. O(n log n).

DRY-RUN THESE CASES

[1,3,4,2,6,8] gives [1,3,4]. An odd length. [0,0]. [0,0,0] fails. Negatives such as [-2,-4,1,2].

LIKELY FOLLOW-UPS

Linear time when the value range is small (counting array), or unsorted output.

PITFALLS

INTERVIEW-DAY PLAN

Sorting by value instead of absolute value for negatives. Mishandling zeros.

HOW TO THINK ABOUT IT

The element with the smallest absolute value cannot be the double of anything, so it is an original; its double must be present too. Pair them off, take the next smallest, and so on. Process values in order of absolute value so negatives and zeros follow the same rule, and handle zero separately (it is its own double, so it needs an even count).

Basis: LeetCode 2007, Find Original Array From Doubled Array.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(changed):
    items = sorted(changed, key=abs)
    originals = []
    while items:
        x = items.pop(0)
        if 2 * x not in items:
            return []
        items.remove(2 * x)
        originals.append(x)
    return originals
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

list.remove is a linear search, so each step is O(n) and the loop is O(n^2). A Counter processed in order of absolute value is O(n log n).

OPTIMAL SOLUTION · Counter processed in order of absolute value

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

FOLLOW-UP 2 Values are small non-negative integers.

Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

FOLLOW-UP 3 Why does processing by absolute value work with negatives?

For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reconstruct original array from doubled
# shuffle
#
# This question evaluates array manipulation,
# multiset/frequency reasoning, pairing logic, and careful
# handling of edge cases such as zeros and negative
# numbers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the originals in the order they appear in the
# input.
# 2. Values are small non-negative integers.
# 3. Why does processing by absolute value work with
# negatives?
#
# INTERVIEW NOTES
# The element with the smallest absolute value cannot be
# the double of anything, so it is an original; its double
# must be present too. Pair them off, take the next
# smallest, and so on. Process values in order of absolute
# value so negatives and zeros follow the same rule, and
# handle zero separately (it is its own double, so it
# needs an even count).
#
# BRUTE FORCE (say it first): Try to remove each smallest
# element together with its double from a list
# Sort by absolute value and, for each element still in
# the list, remove it and one copy of its double
# (list.remove is a linear search). Correct and short; the
# repeated linear searches make it quadratic. Time O(n^2).
# Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Try to remove each smallest
# element together with its double from a list | time
# O(n^2)
def original_brute(changed):
    if len(changed) % 2:
        return []
    rest = sorted(changed, key=lambda x: (abs(x), x))
    out = []
    while rest:
```

```
        x = rest.pop(0)
        if 2 * x not in rest:
            return []
        rest.remove(2 * x)
        out.append(x)
    return out


# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[x]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times.
def find_original(changed):
    # `changed` holds every original x and its double 2x,
    # in
    # any order. Smallest absolute value first: it can
    # only be
    # an original, never someone's double.
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            # zeros pair with each other
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        # every x needs its own 2x
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out


# FOLLOW-UP 1: Return the originals in the order they
# appear in the input.
# Answer: Compute the multiset of originals, then scan the
```

```
# input once and keep each element while its remaining
# original count is positive.
def find_original_in_order(changed):
    # originals in the order they appear in the input
    originals = find_original(changed)
    if not originals and changed:
        return []
    left = Counter(originals)
    out = []
    for x in changed:
        if left[x] > 0:
            left[x] -= 1
            out.append(x)
    return out


# FOLLOW-UP 2: Values are small non-negative integers.
# Answer: Use a counting array instead of sorting and a
# Counter: process values from small to large, O(n +
# range).
def find_original_small(changed, limit):
    # values 0..limit: a counting array, O(n + limit)
    if len(changed) % 2:
        return []
    count = [0] * (2 * limit + 2)
    for x in changed:
        count[x] += 1
    out = []
    if count[0] % 2:
        return []
    out += [0] * (count[0] // 2)
    for x in range(1, limit + 1):
```

```
        if count[x]:
            if count[2 * x] < count[x]:
                return []
            out += [x] * count[x]
            count[2 * x] -= count[x]
    return out


if __name__ == "__main__":
    got = sorted(find_original([1, 3, 4, 2, 6, 8]))
    assert got == [1, 3, 4]
    assert find_original([6, 3, 0, 1]) == []
    assert find_original([1]) == []
```

```
    assert find_original([0, 0, 0, 0]) == [0, 0]
    assert find_original([0, 0, 0]) == []
    # negative numbers: -2 is the double of -1
    neg = sorted(find_original([-1, -2, 4, 2]))
    assert neg == [-1, 2]
    ordered = find_original_in_order([4, 1, 2, 8, 2, 4])
    assert ordered == [4, 1, 2]
    small = sorted(find_original_small([1, 2, 4, 8], 8))
    assert small == [1, 4]
    assert find_original_small([1, 3], 3) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(changed: list[int]) -> list[int]:
```

Sample call

```
solution([2, 1, 4, 8])
```

Test cases

Input	Expected
`([2, 1, 4, 8],)`	`[1, 4]`
`([-4, -2, -2, -1, 1, 2],)`	`[-2, -1, 1]`

STUDY PATH: question 98 of 167 (more practice)

Priority score 1.5 | Hash maps and sets | asked 182 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 5 of 5

Gold Chain Split into Two Equal Halves

ASKED 2026-04-21 | 182 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 201/228

ABOUT THIS QUESTION

Topic: More practice - Hash maps and sets

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEArrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

APPROACH

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.

Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.

Watch the boundary: remove + cut should not yield an empty side.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether removing a segment keeps the rest as a chain joined end to end, whether the cut after removal must leave two non-empty parts, and whether to return a boolean, one answer, or all valid pairs.

BRUTE FORCE TO SAY OUT LOUD

For each removed index, rebuild the array and try every cut with prefix sums. O(n^2).

OPTIMAL IDEA

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

After removing segment r the target half is (S - a[r]) / 2, and a cut means some prefix of the remaining chain has exactly that weight. A prefix of the remaining chain is either an ordinary prefix of the original array (stops before r) or an original prefix minus a[r] (passes over r). Each is one value to look up in a hash of prefix sums, so every r costs O(1).

Basis: Written; prefix sums with a hash lookup (like LeetCode 724 and 560). Assumed rule (confirm it): remove exactly one segment, reconnect the remaining chain end to end, then cut it once into two non-empty parts of equal total weight.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Gold Chain Split into Two Equal Halves
#
# Onsite coding interview 1: given a gold chain
# represented by an array of segment weights, remove
# exactly one segment and check whether the remaining
# segments — when reconnected — can be cut once more to
# give two halves of equal total weight. Follow-up:
# enumerate all valid (remove, cut) pairs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. List every valid (removed segment, cut position)
# pair.
# 2. The chain is a circle (cut anywhere).
# 3. Remove k segments instead of one.
#
# INTERVIEW NOTES
# After removing segment r the target half is (S - a[r]) /
# 2, and a cut means some prefix of the remaining chain
# has exactly that weight. A prefix of the remaining chain
# is either an ordinary prefix of the original array
# (stops before r) or an original prefix minus a[r]
# (passes over r). Each is one value to look up in a hash
# of prefix sums, so every r costs O(1).
#
# BRUTE FORCE (say it first): For each removal rebuild the
# chain and try every cut
# Remove each segment, then test every cut position with a
# running sum. Correct, with a quadratic loop that is the
# baseline to improve. Time O(n^2). Space O(n).


# BRUTE FORCE (say it first): For each removal rebuild the
# chain and try every cut | time O(n^2)
def gold_chain_brute(a):
    n = len(a)
    for r in range(n):
        rest = a[:r] + a[r + 1 :]
        total = sum(rest)
        run = 0
        for c in range(len(rest) - 1):
            run += rest[c]
            if run * 2 == total:
                return True
    return False


# HELPERS for the base question
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P


# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[j] be the sum of the first j segments.
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False


# FOLLOW-UP 1: List every valid (removed segment, cut
# position) pair.
# Answer: The answer itself can be quadratic, so a simple
# loop is acceptable.
def valid_pairs(a):
    # follow-up: every (removed index, cut position) pair;
    # the answer can be quadratic, so a loop is fine
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total, run = sum(rest), 0
        # cut after `cut` segments of the reconnected
```

```
        # chain
        for cut in range(1, len(rest)):
            run += rest[cut - 1]
            if run * 2 == total:
                pairs.append((r, cut))
    return pairs


if __name__ == "__main__":
    # remove the middle 1, then cut 5 | 5
    assert gold_chain([5, 1, 5])
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])
    assert (1, 1) in valid_pairs([1, 2, 1])
    # both methods agree on many small chains
    import itertools
    for chain in itertools.product(range(1, 5), repeat=5):
        assert gold_chain(list(chain)) == bool(
            valid_pairs(list(chain)))
```

NOTES FROM CANDIDATES

Precompute prefix sums; for each candidate removed index r, the remaining total is S - a[r]. If that is odd, skip. Otherwise check if there is a split point achieving (S - a[r]) / 2.
Use a hashmap from prefix-sum value to index for O(1) lookups -> overall O(N) for the boolean answer, O(N^2) worst case for enumerating all pairs (output-bound).
Watch out for cut positions adjacent to the removed link — must still respect contiguous-chain semantics.

PREPARATION

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.